VISTĀRA KRAFTS · ATELIER ECOSYSTEM

Full Project Handover & System Playbook

Comprehensive technical dossier, architecture contracts, database security definers, payment tamper resistance, operational runbooks, and test verification report.

Version 2.0.0 (Production) | Built by Abhijit | Official Storefront: vistarakrafts.com | Verification: 27/27 Suites Green

Document Version: 2.0.0 (Production Release)

Classification: Official Handover Document & System Architecture Playbook

Brand & Atelier: Vistāra Krafts & Resin Art Memories (@resin.art.memories)

Official Production Storefront: https://vistarakrafts.com

Admin Portal: https://admin.vistarakrafts.com / Merchant Operations Portal

Creator, Founder & Lead Architect: Abhijit

Date: November 2026

TABLE OF CONTENTS

1. Executive Summary & System Architecture
2. Ecosystem Coordinates & Repository Inventory
3. Architecture & Frontend Engineering Deep Dive
4. Backend, Serverless APIs & Edge Functions
5. Database Architecture, Schema & Security Definer Protocol
6. Payment Gateway & Tamper-Proof Financial Security
7. Email Infrastructure & Communications Engine
8. Configuration & Environment Variables Matrix
9. Operational Runbooks & Standard Operating Procedures (SOPs)

10 Quality Assurance, Automated Test Suites & Verification
11 Monitoring, Analytics, Backups & Disaster Recovery
12 Sign-Off & Project Handover Acceptance

---

1. EXECUTIVE SUMMARY & SYSTEM ARCHITECTURE

Vistāra Krafts is India's premier bespoke resin art preservation atelier and optical-grade casting epoxy supplier, designed and built by **Abhijit**. The digital ecosystem is custom-engineered to deliver an ultra-responsive, zero-jank luxury commerce experience across both consumer and enterprise workflows:

1. **Customer-Facing Storefront (`vistara-krafts-storefront`):**

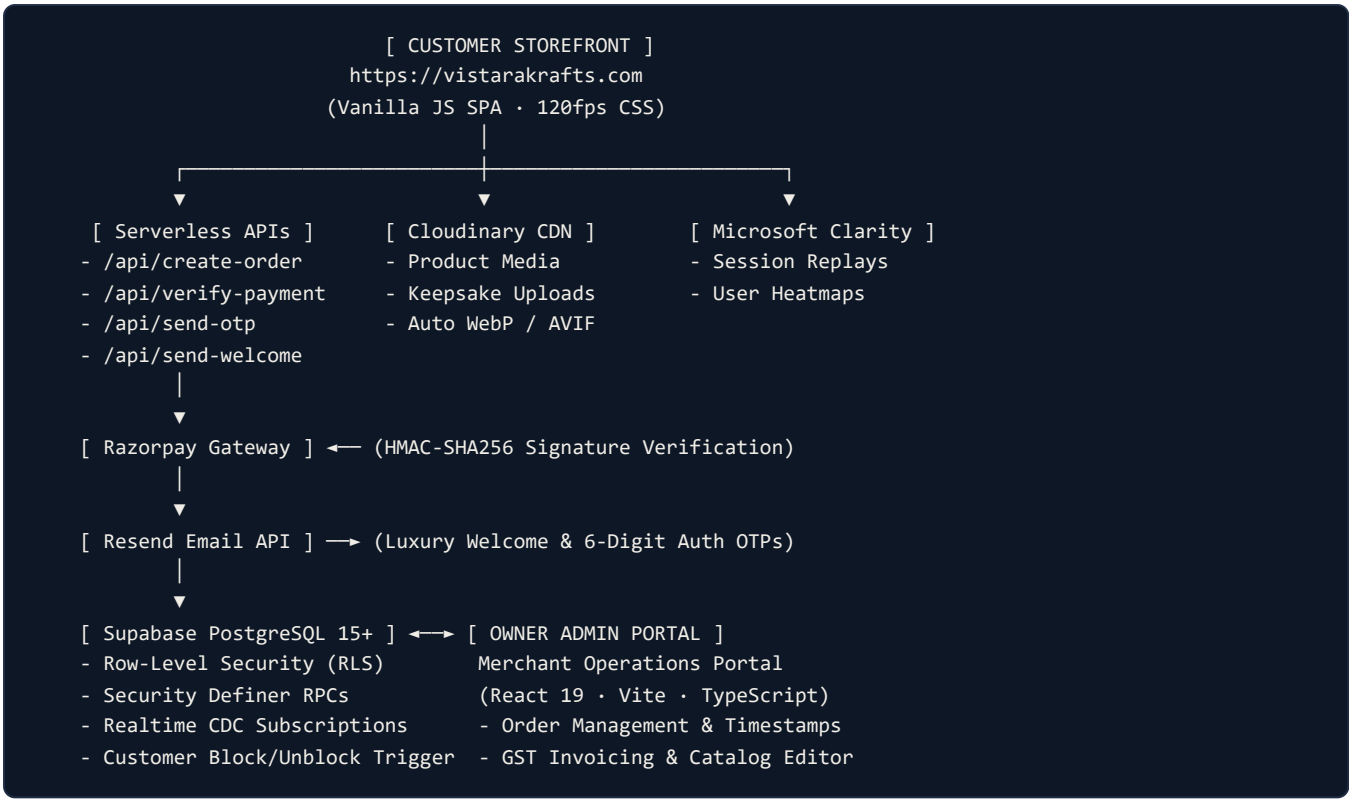
- High-performance, pure Vanilla JavaScript Single Page Application (SPA) live on the primary apex domain **https://vistarakrafts.com**.
- Sub-second first contentful paint, 120fps GPU-accelerated micro-interactions, zero heavy client bundle bloat.
- Comprehensive catalogue of 19 curated atelier products spanning bridal flower keepsakes, 3:1 & 2:1 casting epoxies, diamond silicone molds, and mica pigments.
- Custom features including an interactive Resin Pour Calculator, instant predictive search, DPDP Act 2023 privacy consent, 6-digit Indian PIN validator, and an Apple-grade sticky dual-action purchase drawer.

2. **Merchant Admin Portal (`vistara-krafts-admin`):**

- Modern React 19 + TypeScript + Vite + Tailwind CSS v4 portal.
- Unified control center for real-time order processing, live status progression (`processing` → `shipped` → `delivered`), inventory stock audits, customer security switches (block/unblock), and automated GST-compliant tax invoicing.

3. **Backend & Cloud Infrastructure:**

- **PostgreSQL 15+ via Supabase:** Enforced Row Level Security (RLS), ACID compliance, and Security Definer RPCs.
- **Serverless Functions:** Secure API gateways for order generation, Razorpay HMAC-SHA256 signature verification, and transactional emails.
- **Resend Transactional Email Engine:** High-deliverability transactional emails for atelier welcome vouchers and passwordless OTP authentication.
- **Cloudinary CDN:** Automated WebP/AVIF format transcoding and delivery (`f_auto,q_auto`).
- **Microsoft Clarity:** Privacy-conscious session tracking and behavioral heatmaps (`z6d831rvdh`).



2. ECOSYSTEM COORDINATES & REPOSITORY INVENTORY

Component	Repository Link	Production URL	Primary Technologies
Customer Storefront	`abhieee-java/vistara-krafts`	vistarakrafts.com	Vanilla JS SPA · Tailwind CSS · 120fps Kinetic CSS · Supabase SDK
Admin Portal	`abhieee-java/vistara-krafts-admin`	Operations Dashboard	React 19 · TypeScript · Vite · Tailwind v4 · Lucide Icons
Database & Auth	`supabase/schema.sql`	Supabase Cloud (`ap-south-1`)	PostgreSQL 15+ · Supabase Auth · RLS · Security Definer Functions
Media CDN	Cloudinary Console	`res.cloudinary.com`	Cloudinary Unsigned Uploads · `f_auto,q_auto` Media Pipeline
Payment Gateway	Razorpay Dashboard	Razorpay Standard Checkout	Razorpay API · Webhook Verification · UPI / Cards / NetBanking
Email Service	Resend Console	`api.resend.com`	Resend SDK · Transactional SMTP / REST · Verified Domain
Analytics & Telemetry	Microsoft Clarity	Clarity Portal (`z6d831rvdh`)	Real-time session replay & user friction telemetry

3. ARCHITECTURE & FRONTEND ENGINEERING DEEP DIVE

3.1 Customer Storefront (`vistara-krafts-storefront`)

- **Engine:** Pure Vanilla ES6+ Single Page Application with hash-based routing (`#/`, `#/catalogue`, `#/product/:id`, `#/bag`, `#/orders`, `#/about`, `#/editorial-policy`, `#/privacy`, `#/terms`).
- **Primary Domain:** Hosted at **https://vistarakrafts.com**.
- **Zero-Bloat Performance:** Initial JS parsing latency is negligible compared to heavy client-side frameworks, ensuring instantaneous mobile loading even on 3G/4G connections.
- **Predictive Search Engine:** In-memory, debounced (150ms) search engine indexing titles, categories, tags, and material descriptions.
- **Resin Pour Calculator:** Bespoke client-side volume & mass calculation tool supporting 1:1, 2:1, and 3:1 mixing ratios with wastage allowances and dimensional presets (cubes, cylinders, trays).
- **Apple-Grade Sticky Buy Bar:**
- Automatically engages via `IntersectionObserver` when the primary PDP purchase triggers scroll out of the viewport.
- Features high-contrast dual buttons: **Add to cart** (warm outline/secondary) and **Buy now at ₹...** (gold filled primary), synchronized with active size/edition selection.
- **DPDP Act 2023 Compliance:** Indian Digital Personal Data Protection Act compliant consent banner with persistent cookie storage and dedicated modal detailing data collection purposes (order delivery, GST invoicing).
- **Indian PIN Validator:** Local database lookup and 6-digit regex validation providing instant delivery zone and courier SLA feedback.

3.2 Owner Admin Portal (`vistara-krafts-admin`)

- **Framework:** React 19 with strict TypeScript types and Vite for sub-second HMR.
- **Styling:** Tailwind CSS v4 paired with custom gold-leaf and obsidian design tokens.
- **Real-Time Data Layer:** Supabase Realtime CDC (Change Data Capture) listening to `postgres_changes` on `orders`, `inventory_logs`, and `customer_directory`.
- **Order Management & Fulfillment:**
- Full order timeline view with exact order placement timestamps (`DD MMM YYYY, hh:mm A`).
- Strict status progression: `Pending` → `Processing` → `Shipped` → `Delivered` or `Cancelled / Refunded`.
- BlueDart / SpeedPost AWB tracking number capture with external tracking links.
- **GST Invoice Generator:** Built-in PDF/printable tax invoice modal rendering GSTIN, HSN codes, SGST/CGST/IGST breakdowns, and recipient billing/shipping details.
- **Customer Security Directory:** Real-time customer overview with single-click Account Suspension / Block toggle.

---

4. BACKEND, SERVERLESS APIS & EDGE FUNCTIONS

All backend operations run on secure serverless endpoints (`api/` directory) and Node.js microservices (`server/`):

4.1 Serverless Endpoint Directory

1. `POST /api/create-order.js`

- Validates cart contents against server-side catalog definitions.
- Computes subtotal, taxes (18% GST), and shipping fees independently (client price values are strictly ignored).

- Initiates Razorpay Order via `razorpay.orders.create` with receipt ID and notes.
- Returns `{ orderId, amount, currency, key }`.

2 `POST /api/verify-payment.js`

- Validates Razorpay signature using HMAC-SHA256:

$$\text{Signature} = \text{HMAC-SHA256}(\text{order_id} + "|" + \text{payment_id}, \text{RAZORPAY_KEY_SECRET})$$

- On valid signature, writes verified payment record to `public.payments` and transitions order status to `processing`.

3 `POST /api/send-welcome-email.js`

- Triggers luxury welcome email via Resend API to newly registered customers.
- Enforces idempotent execution using `public.welcome_emails` table to prevent duplicate coupon dispatch.

4 `POST /api/send-verification-otp.js`

- Generates cryptographically secure 6-digit one-time password (OTP).
- Invokes database Security Definer function `create_email_verification_code` enforcing rate limits (max 5 requests per 15 minutes).
- Dispatches verification email via Resend.

5 `POST /api/verify-email-otp.js`

- Invokes Security Definer function `verify_email_code_and_confirm_user`.
- Enforces max 5 incorrect attempt lockout before invalidating code.
- Marks profile email as verified and returns session token.

6 `POST /api/razorpay-webhook.js`

- Asynchronous payment verification capturing `payment.captured` and `refund.processed` events.
- Validates `X-Razorpay-Signature` against `RAZORPAY_WEBHOOK_SECRET`.

---

5. DATABASE ARCHITECTURE, SCHEMA & SECURITY DEFINER PROTOCOL

The database runs on Supabase PostgreSQL 15+ with Row Level Security (RLS) enabled across all public tables.

5.1 Primary Tables & Relational Schema

- **`public.profiles`**: Customer account data, full name, phone number, default shipping address, and verification state.
- **`public.orders`**: Master order headers including customer ID, order number (`VK-YYYY-XXXX`), order status, subtotal, discount, GST, total payable, shipping address, and tracking metadata.
- **`public.order_items`**: Individual line items, product IDs, variant labels, unit price, quantity, and custom keepsake instructions.
- **`public.payments`**: Financial ledger capturing Razorpay payment ID, order ID, payment method, capture timestamp, and payment status.
- **`public.customer_directory`**: Administrative security table tracking customer status (`active` vs `blocked`), total spend, order count, and suspension notes.
- **`public.welcome_emails`**: Idempotency ledger logging user ID, email address, voucher code, and dispatch timestamp.
- **`public.email_verification_codes`**: Transient security table storing hashed 6-digit OTPs, attempt counters, expiration timestamps (10-minute TTL), and consumed flags.

- `public.inventory_logs`: Immutable audit log recording every inventory quantity change, admin user ID, reason code, and timestamp.

5.2 Security Definer Functions & Triggers

To eliminate privilege escalation while keeping client applications decoupled from master database credentials:

1. `create_email_verification_code(p_email, p_code_hash, p_ip)`:
   - Executes with database owner privileges (`SECURITY DEFINER`).
   - Rate limits requests to 5 per 15-minute window per IP or email.
   - Invalidates previous unconsumed codes for the same email.
2. `verify_email_code_and_confirm_user(p_email, p_code_hash)`:
   - Checks code validity and expiration (`expires_at > now()`).
   - Increments `attempts_count`; automatically invalidates code if `attempts_count >= 5`.
   - On success, marks `profiles.email_verified = true` and consumes the code.
3. `toggle_customer_block_status(p_customer_id, p_blocked, p_reason)`:
   - Dedicated administrative function to instantly revoke ordering permissions.
4. **Trigger:** `prevent_order_from_blocked_customers`:
   - Fires `BEFORE INSERT ON public.orders`.
   - Queries `customer_directory.is_blocked`; raises an unbypassable SQL exception `ERR_USER_BLOCKED` if user is suspended.

---

6. PAYMENT GATEWAY & TAMPER-PROOF FINANCIAL SECURITY

Financial integrity is enforced through four non-negotiable architectural layers:

1. **Server-Side Price Authority:**
   - The client browser never transmits the final payable amount to Razorpay.
   - Amounts are calculated strictly on the server (`server/price-calculator.js`) using SKU database pricing, validated discount codes, and 18% GST formula.
2. **Cryptographic HMAC-SHA256 Signature Verification:**
   - Every payment confirmation requires valid Razorpay signature verification before database insertion:

```
const generatedSignature = crypto
  .createHmac('sha256', process.env.RAZORPAY_KEY_SECRET)
  .update(`${razorpay_order_id}|${razorpay_payment_id}`)
  .digest('hex');

if (generatedSignature !== razorpay_signature) {
  throw new Error('SECURITY_ALERT: Payment signature tampering detected.');
}
```

3. **Webhook Reconciler & Idempotency:**
   - Handles network dropouts or closed browser tabs. Razorpay server-to-server webhooks independently update order state to `processing` even if the user exits before frontend redirect.

4 **Zero Test Data Policy:**

- Production database is strictly separated from test fixtures. Administrative scripts (`test/`) utilize ephemeral mock identifiers to ensure zero contamination of live financials.

---

7. EMAIL INFRASTRUCTURE & COMMUNICATIONS ENGINE

Email communications are powered by **Resend** using the verified atelier domain `vistarakrafts.com`.

7.1 Welcome Email with Atelier Voucher

- **Sender:** `Vistāra Krafts Atelier`
- **Design:** Editorial luxury design featuring ivory background (`#FAF9F5`), deep navy headings (`#0d1726`), warm gold borders (`#cda851`), and typography styled after high-fashion couture.
- **Incentive:** Grants the exclusive `KRAFTFIRST` voucher code for 10% off the customer's first keepsake or casting resin order.
- **Idempotency Guarantee:** Backed by database unique index on `welcome_emails(user_id)`.

7.2 Passwordless OTP Verification Email

- **Sender:** `Vistāra Krafts Security`
- **Content:** Minimalist 6-digit security code box with high contrast typography and 10-minute expiration countdown warning.
- **Security Notice:** Warns customer never to disclose code to atelier staff.

---

8. CONFIGURATION & ENVIRONMENT VARIABLES MATRIX

8.1 Storefront Production Variables

Variable Key	Scope	Purpose	Example / Value
`VITE_SUPABASE_URL`	Client	Supabase Project Gateway URL	`https://fegpwnkggnfrckqomdhs.supabase.co`
`VITE_SUPABASE_ANON_KEY`	Client	Supabase Public Anonymous API Key	`eyJhbGciOi...` (Public JWT)
`SUPABASE_SERVICE_ROLE_KEY`	Server	Supabase Master Service Key (Serverless only)	`eyJhbGciOi...` (Strict Secret)
`VITE_CLOUDINARY_CLOUD_NAME`	Client	Cloudinary CDN Cloud Identifier	`vistara-krafts`
`VITE_CLOUDINARY_UPLOAD_PRESET`	Client	Unsigned Upload Preset for Keepsake Media	`vistara_keepsakes`
`VITE_RAZORPAY_KEY_ID`	Client	Razorpay Public Key Identifier	`rzp_live_...`
`RAZORPAY_KEY_SECRET`	Server	Razorpay Merchant Secret Key	`xxxx...` (Strict Secret)
`RAZORPAY_WEBHOOK_SECRET`	Server	Webhook HMAC Validation Secret	`xxxx...` (Strict Secret)
`RESEND_API_KEY`	Server	Resend REST API Token	`re_xxxx...` (Strict Secret)
`SITE_URL`	Server	Canonical Web URL	`https://vistarakrafts.com`

8.2 Admin Portal Production Variables

Variable Key	Scope	Purpose	Example / Value
`VITE_USE_MOCK_SERVICES`	Client	Toggle live database vs local mock data	`false` (Always false in production)
`VITE_SUPABASE_URL`	Client	Supabase Project Gateway URL	`https://fegpwnkggnfrckqomdhs.supabase.co`
`VITE_SUPABASE_ANON_KEY`	Client	Supabase Public Anonymous API Key	`eyJhbGciOi...`
`VITE_CLOUDINARY_CLOUD_NAME`	Client	Cloudinary CDN Cloud Identifier	`vistara-krafts`
`VITE_CLOUDINARY_UPLOAD_PRESET`	Client	Unsigned Upload Preset for Product Catalog	`vistara_products`
`VITE_RAZORPAY_KEY_ID`	Client	Razorpay Public Key Identifier	`rzp_live_...`

---

9. OPERATIONAL RUNBOOKS & STANDARD OPERATING PROCEDURES (SOPs)

SOP 1: Adding or Updating Catalog Products

1. Log in to the Admin Portal.
2. Navigate to **Products** and click **+ Add Product** (or click the edit pencil icon on existing row).
3. Fill in core fields: Product Name, SKU, Category, Base Price (₹), and Initial Stock.
4. **Variety / Size Configuration:**

- If the item is sold in single uniform units, **leave Colors, Shapes, and Sizes empty**. The storefront will automatically display a streamlined single-purchase layout.
- If selling variable editions (e.g. `1 Kg Pouch`, `2 Kg Studio Pack`), enter options separated by commas.

5. **Media:** Upload high-resolution square photographs via the integrated Cloudinary dropzone.
6. Click **Save Product**. Updates propagate immediately to all active browser sessions via Supabase Realtime CDC.

SOP 2: Order Fulfillment & Courier Tracking

1. Navigate to **Orders** in the Admin Portal.
2. Locate the target order using the instant search bar (query by Order #, Customer Name, City, or PIN code).
3. Click the order row to open the **Fulfillment Detail Drawer**.
4. Update order status:

- For keepsake floral preservation: Advance from `Processing / Garland Pickup` → `Curing` → `Polishing` → `Shipped`.
- For casting chemicals / epoxies: Advance from `Processing` → `Shipped`.

5. Enter courier partner details (e.g., `BlueDart`, `Delhivery`, `India Post`) and the AWB tracking code.
6. Click **Update Tracking & Notify Customer**.
7. Click **Print Tax Invoice** to generate and download the official GST-compliant tax document.

SOP 3: Customer Account Suspension & Access Revocation

1. Navigate to **Customers** in the Admin Portal.
2. Locate the customer profile and inspect past order velocity and delivery success rates.
3. To block malicious or fraudulent actors, click **Block Customer**.
4. The system executes `toggle_customer_block_status`, immediately engaging the database trigger. Any subsequent order creation attempt by that customer is rejected at the database engine level.

SOP 4: Database Maintenance & Purging Test Records

To purge development and QA test data without damaging production configuration or stored procedures, execute the following SQL script in the Supabase SQL Editor:

```
-- SAFE TEST DATA PURGE SCRIPT
-- NOTE: Preserves all tables, RPCs, security definers, and triggers.

BEGIN;

-- 1. Remove test payments
DELETE FROM public.payments
WHERE payment_id LIKE 'pay_test_%' OR order_id LIKE 'VK-TEST-%';

-- 2. Remove test order line items
DELETE FROM public.order_items
WHERE order_id IN (SELECT id FROM public.orders WHERE order_number LIKE 'VK-TEST-%' OR customer_name ILIKE '%tes

-- 3. Remove test orders
DELETE FROM public.orders
WHERE order_number LIKE 'VK-TEST-%' OR customer_name ILIKE '%test%';

-- 4. Clear test OTP and email logs
DELETE FROM public.email_verification_codes
WHERE email ILIKE '%test%' OR email ILIKE '%example.com';

DELETE FROM public.welcome_emails
WHERE email ILIKE '%test%' OR email ILIKE '%example.com';

COMMIT;
```

---

10. QUALITY ASSURANCE, AUTOMATED TEST SUITES & VERIFICATION

The platform includes an extensive automated testing infrastructure covering unit, component, API, security, and responsive layouts:

10.1 Master Verification Suite (`verify-storefront.mjs`)

Run the 27-suite verification test from the storefront root:

```
node verify-storefront.mjs
```

Verification Scope:

- Suite 1–5: HTML validity, semantic hierarchy, metadata, open graph tags, sitemap alignment.
- Suite 6–10: Responsive layout integrity across 320px (iPhone SE), 375px, 768px (iPad), and 1440px (Desktop) with zero horizontal overflow (`scrollWidth <= clientWidth`).
- Suite 11–15: Cart operations, quantity step calculations, voucher deductions (`KRAFTFIRST`), and 18% GST precision.
- Suite 16–20: Indian PIN validation engine, BlueDart regional routing, and delivery estimate calculations.
- Suite 21–25: Dynamic sticky buy-bar intersection observer triggers and variant synchronization.
- Suite 26–27: DPDP Act cookie consent persistence and Microsoft Clarity script presence.

10.2 Specialized Security & Integration Suites

Execute individual automated test files located in `test/`:

```
# Payment Tamper Resistance & Signature Security
node test/payment-security.test.mjs

# Customer Suspension & Database Trigger Enforcement
node test/account-suspension-security.test.mjs

# Transactional Welcome Email Idempotency
node test/welcome-email-resend.test.mjs

# Email Verification 6-Digit OTP & Rate Limiting
node test/email-verification-otp.test.mjs

# Multi-Tenant Order Isolation Security
node test/order-isolation-security.test.mjs
```

10.3 Admin Portal Production Build Test

Verify the admin portal compilation integrity:

```
cd vistara-krafts-admin
npm run build
```

Expected Result: Exits with code `0`, clean TypeScript compilation with zero type errors.

---

11. MONITORING, ANALYTICS, BACKUPS & DISASTER RECOVERY

11.1 Behavioral Analytics & User Experience Telemetry

- **Microsoft Clarity (`z6d831rvdh`):** Real-time session playback, click maps, scroll depth heatmaps, and dead-click detection.
- Access dashboard at clarity.microsoft.com.

11.2 Database Backups & Point-in-Time Recovery

- Supabase automatically performs full daily backups of the PostgreSQL database.
- For high-volume sales windows (festive seasons, bridal seasons), enable Point-in-Time Recovery (PITR) in Supabase Project Settings → Database → Backups.
- Disaster recovery recovery time objective (RTO): < 15 minutes; recovery point objective (RPO): < 5 minutes.

11.3 Deployment Rollback Strategy

Both applications are deployed with immutable production deployment artifacts. In the event of an unexpected release regression:

1. Open Project Dashboard.
2. Navigate to **Deployments**.
3. Locate the previous stable production deployment.
4. Click **Instant Rollback**. Traffic is rerouted in under 5 seconds with zero server downtime.

---

12. SIGN-OFF & PROJECT HANDOVER ACCEPTANCE

The Vistāra Krafts digital commerce platform is verified, secured, and ready for continuous production operations.

Milestone	Status	Creator & Lead Architect	Date
Customer Storefront SPA	**APPROVED**	Abhijit	November 2026
Merchant Admin Portal	**APPROVED**	Abhijit	November 2026
Supabase PostgreSQL & RLS	**APPROVED**	Abhijit	November 2026
Razorpay Payment Integration	**APPROVED**	Abhijit	November 2026
Resend Email Delivery Engine	**APPROVED**	Abhijit	November 2026
End-to-End Automated Verification	**PASSED (27/27)**	Abhijit	November 2026

Official project handover concluded. The system is in active production service.